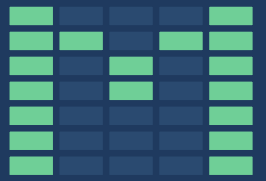
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D



ALGORİTMA VE PROGRAMLAMA – I

H01_K01_merhaba_satiri.c · satır 12

```
12      printf("Merhaba");
```

HAFTA 01

Bilgisayar Nasıl Çalışır? Programın Yaşam Döngüsü ve İlk C Programı

Soyutlama katmanları · Von Neumann · ikili sayılar · derleme hattı · printf

MODÜL I · MAKİNE VE ALGORİTMA

Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 **Bilgisayar ve Program**	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06
H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

BAŞLAMADAN ÖNCE · 3 SORU

Ön bilgi yoklaması: sezgileriniz ne diyor?

Bu ilk hafta; geçen haftayı hatırlamak yerine sezgilerinizi yokluyoruz. Yanıtınızı kâğıda yazın, hafta sonunda karşılaştıracağız.



Soru 1 · Bellek

Fişi çektiğinizde açık belgenin RAM'deki kopyasına ne olur? Diskteki kopyasına? Neden iki ayrı yer var?



Soru 2 · Veri

“A” harfi, bir fotoğraf ve çalışan bir program aynı bellekte nasıl saklanır? Ortak “dilleri” nedir?



Soru 3 · Çeviri

İşlemci C programınızı doğrudan anlar mı? Anlamıyorsa aradaki “çevirmen” kimdir?

Bir satır koddan ekrandaki piksellere

```
H01_K01_merhaba_satiri.c · satır 12
12      printf("Merhaba");
```

→

? derleyici ?

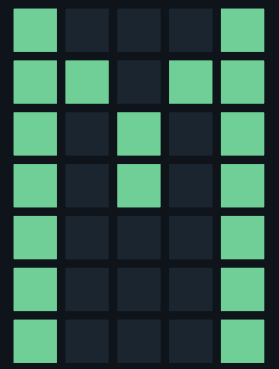
? bağlayıcı ?

? işletim sistemi ?

? sürücü ?

? donanım ?

→



Ekranda her harf bir **piksel** ızgarasıdır

Merhaba, Dünya!

*Klavyede yazdığımız `printf("Merhaba");` satırı, ekranda piksellere dönüşene kadar **hangi katmanlardan** geçer?*

GERÇEK DÜNYADAN

9 Eylül 1947 · Harvard

Kayıt defterine yapıştırılan güve

- Harvard Mark II, binlerce **elektromekanik röleyle** çalışan bir hesap makinesiydi.
- Ekip, arızanın nedenini ararken 70 numaralı rölede sıkışmış bir **güve** buldu ve deftere yapıştırdı: “First actual case of bug being found.”
- “Bug” sözcüğü daha eskiydi (Edison, 1870'ler); olay onu bilgisayar dünyasında **simgeleştirdi**.
- Grace Hopper öyküyü yıllarca anlatarak yaygınlaştırdı; güveyi bulan kişi o değildi. Defter bugün Smithsonian koleksiyonundadır.

Kaynak: National Museum of American History (t.y.), Log book with computer bug, nesne no. 1994.0191.01.

GERÇEK DÜNYADAN

1952 · UNIVAC I

Makineye kendi programını yazdırmak: A-0

- 1950'lerin başında programlar makinenin **sayısal kodlarıyla** elle yazılıyordu.
- Grace Hopper, UNIVAC I için geliştirdiği **A-0 sistemiyle** (1952) alt programları bir kütüphaneden bulup birleştirme işini makineye devretti.
- Fikir: Programı insan yazsın, **makine koduna çevirmeyi başka bir program** yapsın.
- Bu fikir, bu hafta inceleyeceğimiz **derleme hattının** (önişlemci → derleyici → çevirici → bağlayıcı) atasıdır.

Kaynak: Hopper, G. M. (1952). The education of a computer. Proceedings of the 1952 ACM National Meeting.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 1.1 Bilgisayar sistemini **soyutlama katmanları** üzerinden açıklar.

ÖÇ 1.2 Von Neumann mimarisinin bileşenlerini ve **getir–çöz–yürüt** döngüsünü açıklar.

ÖÇ 1.3 Bit, bayt, **ikili ve onaltılı** sayılar arasında dönüşüm yapar.

ÖÇ 1.4 Kaynak koddan çalışan sürece uzanan **derleme hattını** adım adım açıklar.

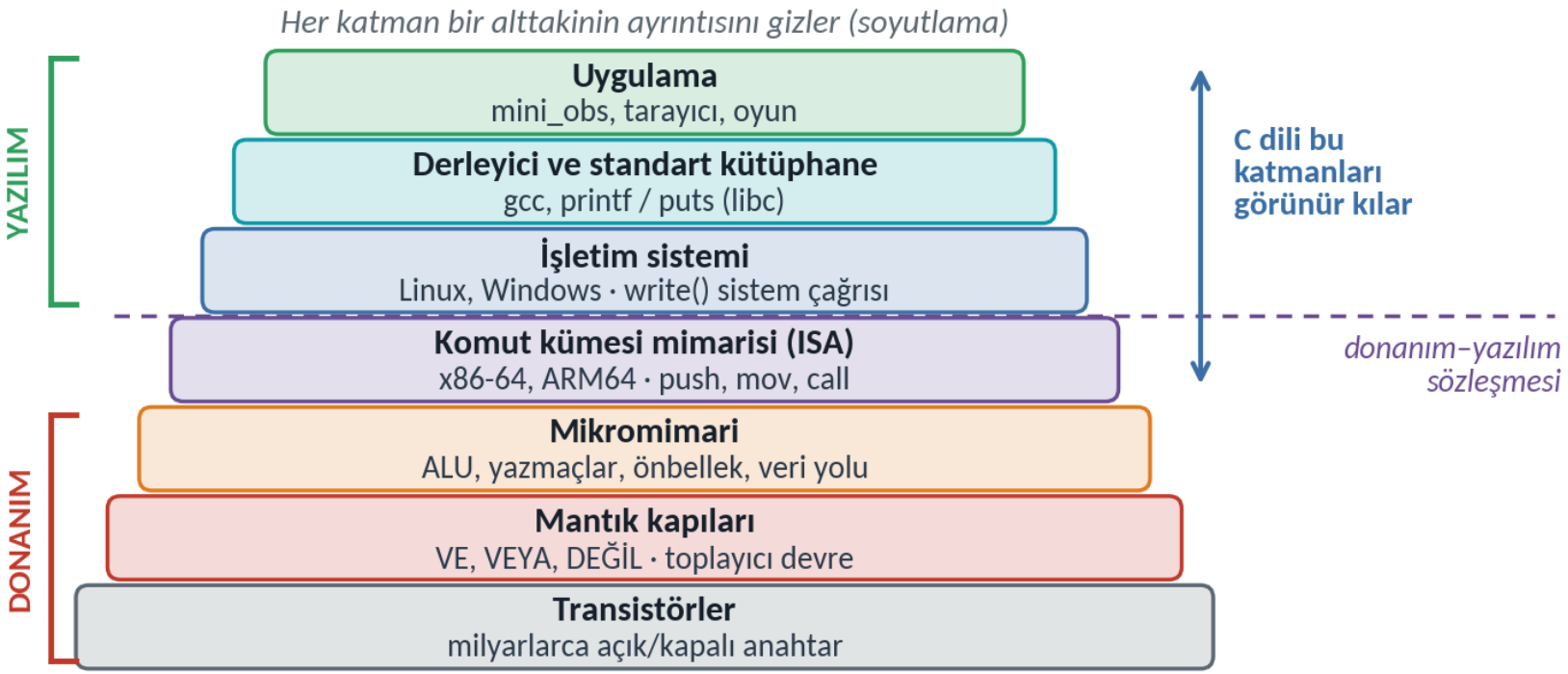
ÖÇ 1.5 Ortamı kurar; ilk programı derler, çalıştırır; **derleyici ve bağlayıcı** hatalarını ayırt eder.

01

Bilgisayar nasıl çalışır?

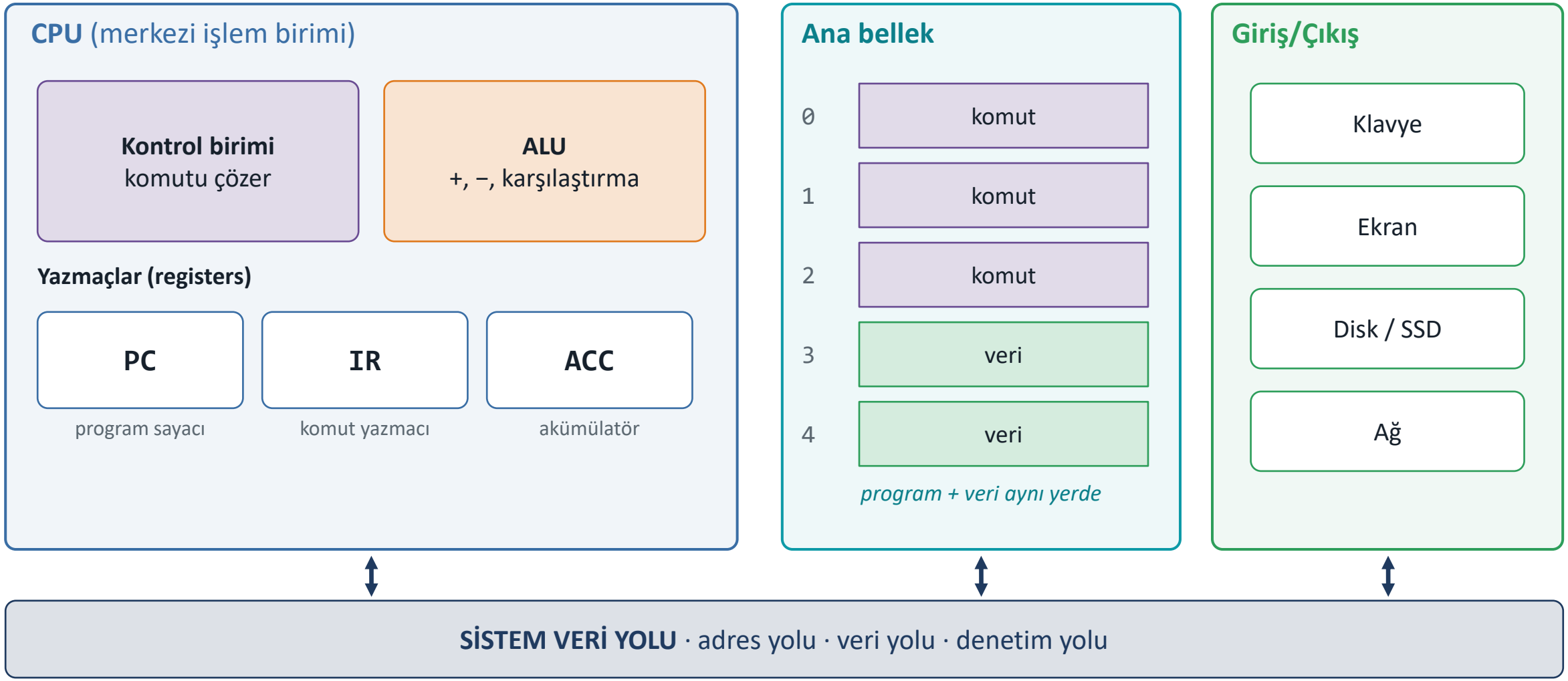
Soyutlama katmanları · Von Neumann · getir–çöz–yürüt · bellek hiyerarşisi

Soyutlama katmanları

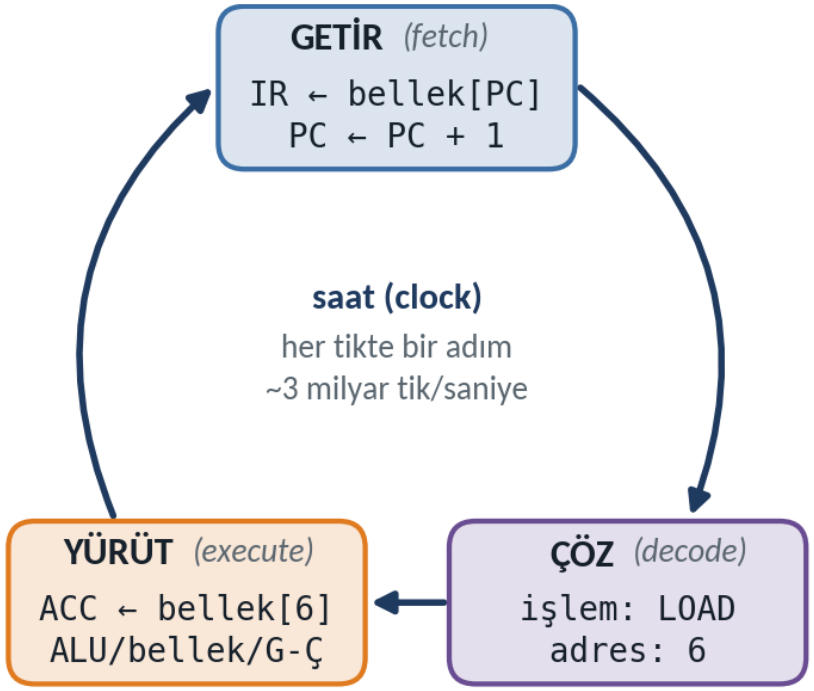


- Her katman **alttakini gizler**, üsttekine basit bir arayüz sunar (pedal ↔ motor).
- **ISA** (x86-64, ARM64): donanım ile yazılım arasındaki **sözleşme**.
- İşletim sistemi kaynaklara **sistem çağrılarıyla** erişim verir.
- C ortadadır: taşınabilir ama belleği gizlemez (“taşınabilir assembly”).

Von Neumann mimarisi



Getir – çöz – yürüt



Örnek: bellek[0] = 16

`PC = 0`	okunacak adres
`IR = 16`	getirilen sayı
`1 → LOAD`	onlar basamağı: işlem
`6 → adres`	birler basamağı: adres
`ACC ← 3`	bellek[6] = 3

Oyuncak komut kümesi

`0x`	`HALT`	dur
`1a`	`LOAD a`	ACC ← bellek[a]
`2a`	`ADD a`	ACC ← ACC + bellek[a]
`3a`	`STORE a`	bellek[a] ← ACC
`4a`	`OUT a`	bellek[a]'yı ekrana yaz
`5a`	`SUB a`	ACC ← ACC − bellek[a]

- **Getir:** IR ← bellek[PC]; PC ← PC + 1
- **Çöz:** işlem kodunu ve adresi ayır
- **Yürüt:** ALU, bellek ya da G/Ç işlemi
- Saat her tikte bir adım: ~3 GHz ≈ **3 milyar tik/sn**
- Oyuncak kodlama: 16 = **LOAD 6**

Oyuncak bilgisayar: 8 hücrelik bellek

PROGRAM (komutlar) · 0–4

VERİ · 5–7

LOAD 6	ADD 7	STORE 5	OUT 5	HALT	(sonuç)	veri	veri
16	27	35	45	00	00	03	04
adres 0	adres 1	adres 2	adres 3	adres 4	adres 5	adres 6	adres 7



PC

0

sıradaki komutun adresi

IR

–

getirilen komut

ACC

0

ara sonuç (akümülatör)

Komut kümesi (işlem kodu + adres)

1a LOAD a: ACC ← bellek[a]
2a ADD a: ACC ← ACC + bellek[a]
3a STORE a: bellek[a] ← ACC
4a OUT a: ekrana yaz · 0x HALT
5a SUB a: ACC ← ACC − bellek[a]

İnsan CPU: programı tur tur yürütelim

oyuncak bellek

```
0: 16   LOAD 6
1: 27   ADD 7
2: 35   STORE 5
3: 45   OUT 5
4: 00   HALT
5: 00
6: 03
7: 04
```

Tur	PC	IR	Komut	ACC	b[5]	Çıktı
1	0→1	16	LOAD 6	3	0	–
2	1→2	27	ADD 7	7	0	–
3	2→3	35	STORE 5	7	7	–
4	**3→4**	**45**	**OUT 5**	**7**	**7**	**7**
5	4→5	00	HALT	7	7	–

TAHMİN ET

bellek[7]'ye 04 yerine 09 yazarsak ekranda ne görünür?

```
oyuncak bellek
0: 16  LOAD 6
1: 27  ADD 7
2: 35  STORE 5
3: 45  OUT 5
4: 00  HALT
5: 00
6: 03
7: 09
```

A 9

B 12

C 7

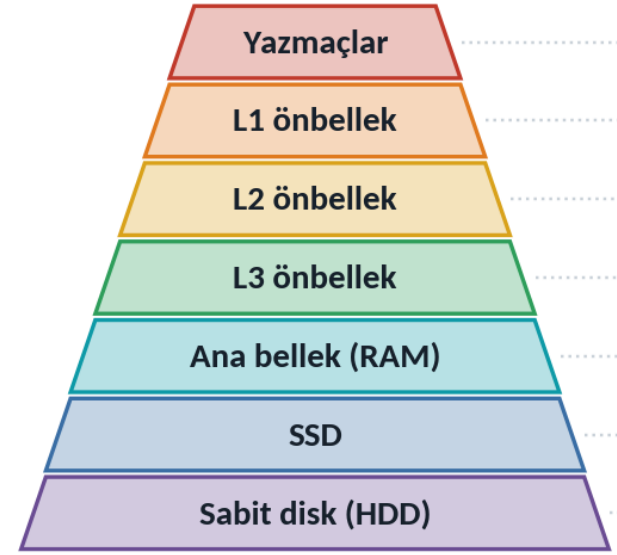
D Program durmaz

ALGORİTMA ATÖLYESİ

Sınıf etkinliği: İnsan CPU

- **Roller:** PC (adres kartı), kontrol birimi (sayıyı işlem + adrese ayırır), ALU/ACC, 8 bellek hücresi, ekran.
- **Tur 1:** Programı yürütün, tahtadaki izleme tablosunu doldurun.
- **Tur 2:** bellek[7] = 9 yapın; önce **tahmin**, sonra yürütme.
- **Tur 3:** Hücre 1'e 57 (SUB 7) yazın: 3 − 4 = ? Negatif sayı nasıl saklanır? → H3
- Web sayfasındaki **Oyuncak CPU** simülasyonuyla tekrar edin.

Bellek hiyerarşisi

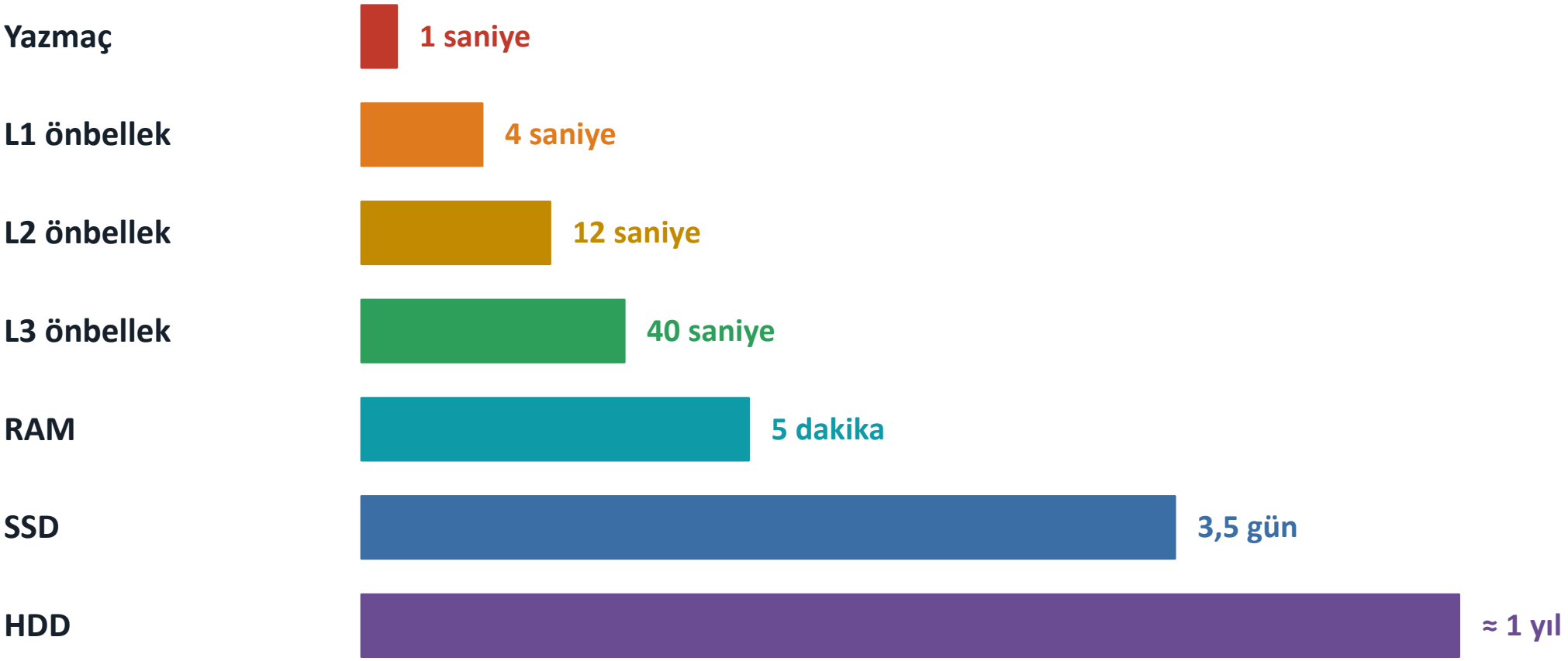


	Kapasite	Erişim süresi	Çevrim	1 çevrim = 1 sn olsaydı
Yazmaçlar	~ yüzlerce bayt	< 1 ns	1	1 saniye
L1 önbellek	32–64 KB	~1 ns	~4	4 saniye
L2 önbellek	0,25–2 MB	~4 ns	~12	12 saniye
L3 önbellek	8–64 MB	~15 ns	~40	40 saniye
Ana bellek (RAM)	8–64 GB	~100 ns	~300	5 dakika
SSD	0,5–4 TB	~100 µs	~300 000	3,5 gün
Sabit disk (HDD)	1–20 TB	~10 ms	~30 000 000	≈ 1 yıl

Tipik mertebeler; donanıma göre değişir (~3 GHz, 1 çevrim ≈ 0,33 ns)

- Hızlı bellek **küçük ve pahalı**, büyük bellek **yavaş ve ucuz**.
- Yazmaç, önbellek, RAM **uçucu**; SSD/HDD **kalıcı**.
- Program diskte durur, çalışırken **RAM'e yüklenir**.
- Değerler mertebe olarak tipik değerlerdir.

1 saat çevrimi = 1 saniye olsaydı…



logaritmik ölçek: çubuk boyu sürenin basamak sayısıyla büyür · ~3 GHz, 1 çevrim ≈ 0,33 ns

Kaputun altında: bu makinenin önbellekleri

TERMİNAL (GERÇEK ÇIKTI, BOŞLUKLAR KISALTILDI)

```
$ lscpu | grep -E 'Model name|L1|L2|L3'
Model name:  Intel(R) Xeon(R) Processor @ 2.10GHz
L1d cache:  96 KiB (2 instances)
L1i cache:  64 KiB (2 instances)
L2 cache:  4 MiB (2 instances)
L3 cache:  260 MiB (1 instance)
```

L1d / L1i: veri ve komut için ayrı L1 (Harvard mimarisi izi)
2 instances: iki çekirdek; her birinde 48 KiB L1d, 2 MiB L2
260 MiB L3: sunucu işlemcisi; dizüstülerde ~8–32 MB



Kendi makinenizde deneyin

Linux / WSL: `lscpu`

macOS: `sysctl hw.l1dcachesize hw.l2cachesize`

Windows: Görev Yöneticisi → Performans → CPU

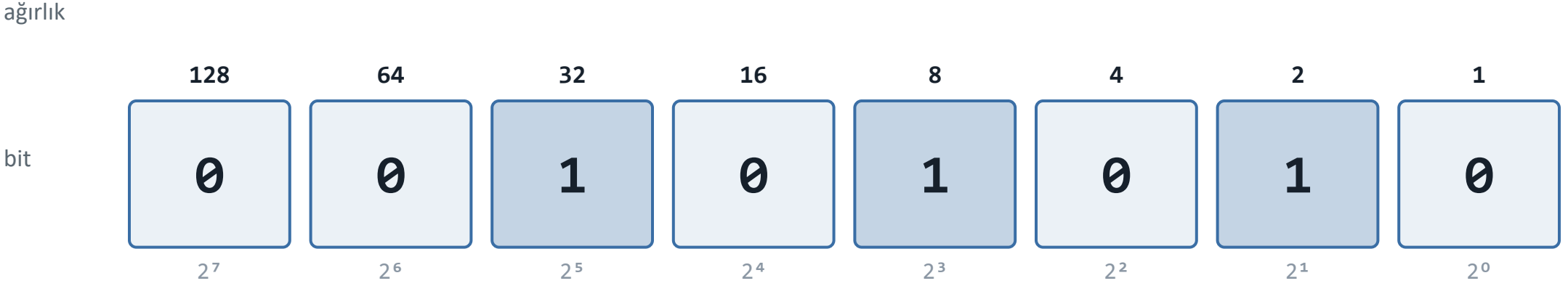
KiB = 1024 bayt · KB = 1000 bayt

02

Veri = bitler

Bit, bayt, ikili ve onaltılı sayılar · ASCII ve UTF-8

Bir bayt = 8 bit



$$32 + 8 + 2 = \mathbf{42}$$

`0010` = **2**

`1010` = **A** (10)

$0010\ 1010_2 = 42_{10} = 0x2A$ · 4 bit (yarım bayt) = 1 onaltılı basamak · 8 bit → 256 değer (0–255)

EZBERLEMEYE DEĞER TABLO

0–15: onluk · ikili · onaltılı

Onluk	İkili	Onaltılı	Onluk	İkili	Onaltılı
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

Dönüşüm yöntemleri

Onluk → ikili: 2'ye böl

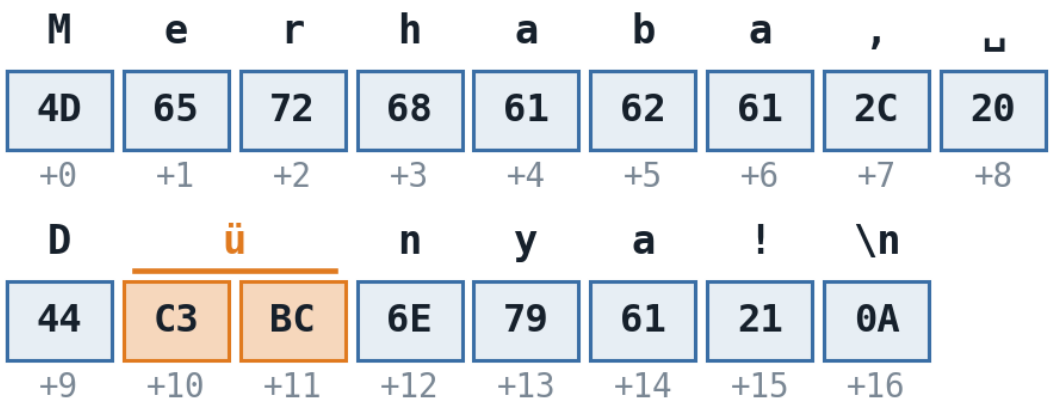
- 42 ÷ 2 = 21 kalan **0**
- 21 ÷ 2 = 10 kalan **1**
- 10 ÷ 2 = 5 kalan **0** · 5 ÷ 2 = 2 kalan **1**
- 2 ÷ 2 = 1 kalan **0** · 1 ÷ 2 = 0 kalan **1**
- Kalanları **aşağıdan yukarı** oku: 101010_2

İkili ↔ onaltılı: 4'lü grupla

- $1100\ 1000_2$ → C 8 → **0xC8**
- Onaltılı → onluk: 0x2A = 2·16 + 10 = **42**
- 0xFF = 15·16 + 15 = **255**
- İkili → onluk: 1 olan bitlerin ağırlıklarını topla
- C17'de 0x standarttır; 0b yazımı C23 ile geldi

Metin de sayıdır: “Merhaba, Dünya!\n” bellekte

Bellekte “Merhaba, Dünya!\n”: 17 bayt (UTF-8, onaltılı)



ü harfi 2 bayttır: C3 BC

ASCII (0–127) harfleri 1 bayt, Türkçe harfler UTF-8'de 2 bayt *(ayrıntı: Hafta 13)*

$M = 4D_{16} = 77_{10} = 0100\ 1101_2$

\n (yeni satır) = $0A_{16} = 10_{10}$

- 'M' = **0x4D** = 77
- '\n' = **0x0A** = 10
- 'A' = 0x41, 'a' = 0x61: yalnızca **bir bit** (32) farklı
- 'ü' UTF-8'de **2 bayt**: C3 BC
- 16 karakter = **17 bayt**

TAHMİN ET

`0x2A` onluk tabanda kaçtır?

A 24

B 42

C 32

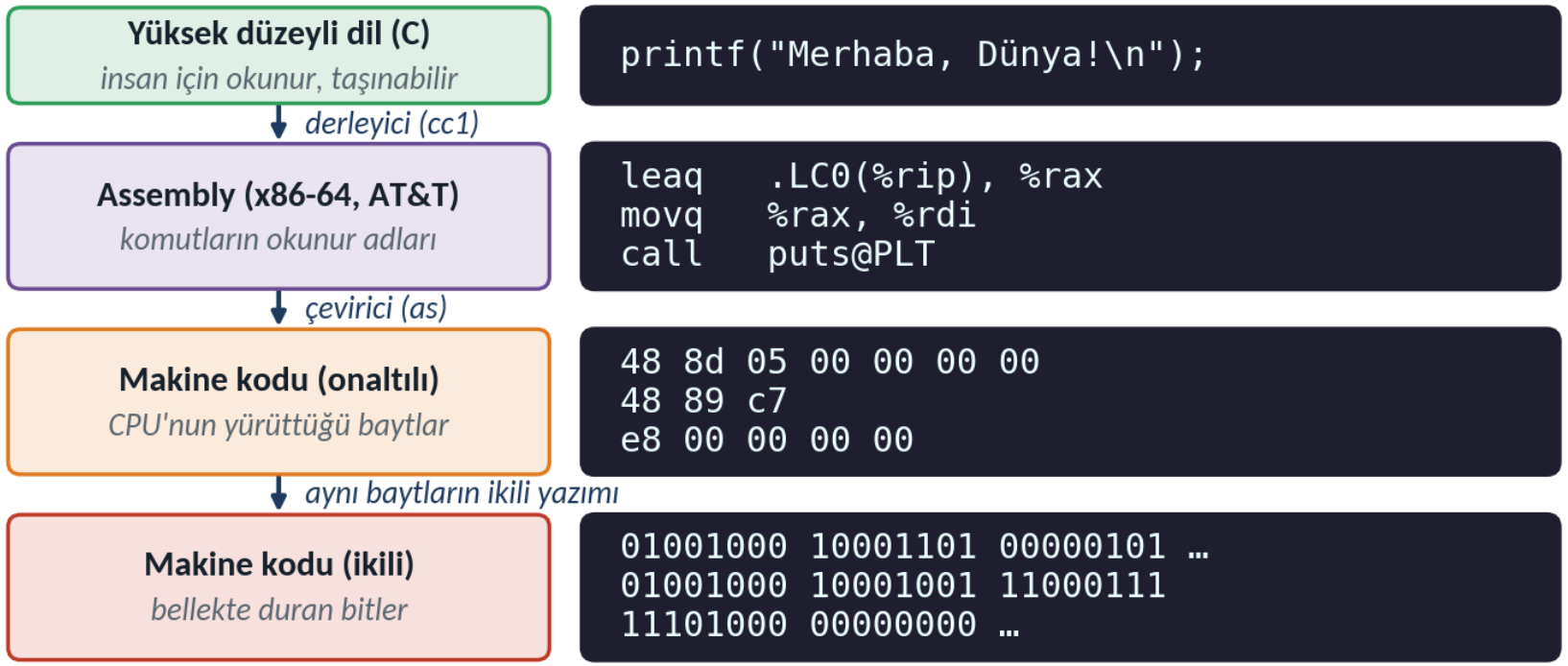
D 210

03

Kaynak koddan makine koduna

Dillerin evrimi · C'nin kısa tarihi · derleme hattı

Aynı işlem dört düzeyde



- İşlemci yalnızca **makine kodunu** yürütür
- **Assembly:** komutların okunur adları; işlemciye özgü
- **C:** işlemciden bağımsız, derleyici çevirir
- Gerçek veri: `gcc -S` ve `objdump -d`

Derlenen ve yorumlanan diller



Derlenen: C, C++, Rust, Go

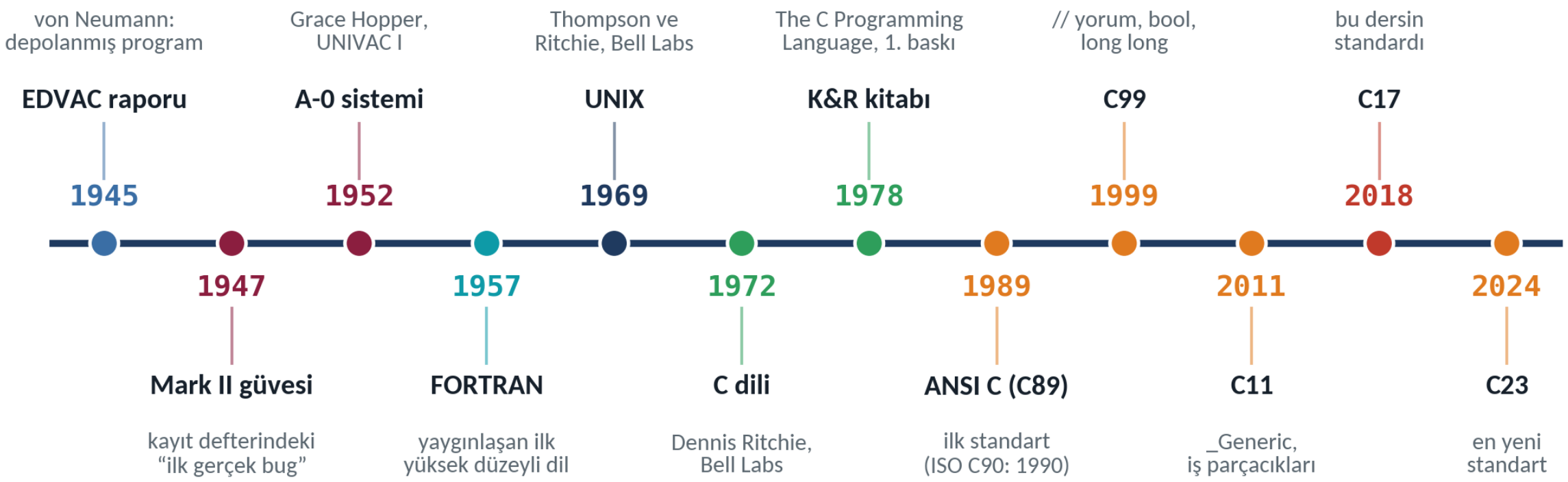
- Çalıştırmadan **önce, bir kez** makine koduna çevrilir
- Genellikle **hızlı**
- Pek çok hata **derlerken** yakalanır
- Çalıştırılabilir dosya yalnızca **derlendiği platformda** çalışır



Yorumlanan: Python, JavaScript

- Yorumlayıcı programı **çalışırken** çevirir
- Genellikle daha yavaş (JIT farkı azaltır)
- Hatalar çoğu zaman **o satır çalışınca** görülür
- Aynı dosya yorumlayıcının olduğu **her yerde** çalışır
- Ara yol: Java → **bayt kodu** → JVM

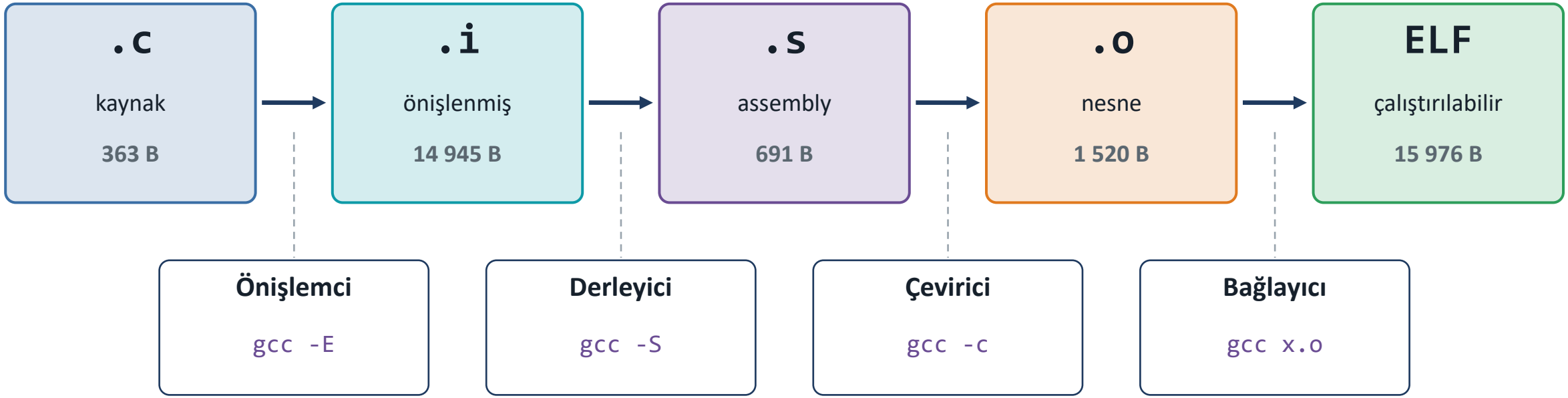
Depolanmış programdan C23'e



Olaylar sıralı gösterilmiştir; yatay eksen ölçekli değildir.

C: Dennis Ritchie, Bell Labs (1972) · UNIX 1973'te büyük ölçüde C ile yeniden yazıldı · bu derste C17 (ISO/IEC 9899:2018)

Derleme hattı: gerçek dosyalar, gerçek boyutlar



`gcc merhaba.c -o merhaba` bu dört aşamayı **tek komutla** çalıştırır; `-save-temps` ara dosyaları saklar. **.i**'nin büyüklüğü: `#include <stdio.h>` metni eklendi (554 satır).

Yükleyici (işletim sistemi): `./merhaba` → dosyayı RAM'e yükler, libc'yi bağlar, `main`'i çalıştırır → **süreç**

Kaputun altında: gcc kimleri çağırıyor?

TERMİNAL (GERÇEK ÇIKTI)

```
$ gcc -v merhaba.c   (özet)
  cc1 → as → collect2
$ ldd ./ara_dosyalar   (adresler gizlendi)
        linux-vdso.so.1
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
        /lib64/ld-linux-x86-64.so.2
$ strace -e trace=write ./ara_dosyalar
write(1, "Merhaba, D\303\274nya!\n", 17) = 17
+++ exited with 0 +++
```

`cc1`: önişlemci + derleyici
`as`: çevirici
`collect2`: bağlayıcı ld'yi çağırır

`libc.so.6`: printf ve puts burada

`write(1, …, 17)`: çekirdeğe yapılan sistem çağrısı; 1 = standart çıktı, 17 bayt

Windows/MinGW: `.exe` (PE), kütüphaneler `.dll` (UCRT64: `ucrtbase.dll`); aşamaların mantığı aynı.

04

İlk C programı ve örnek merdiveni

Kurulum · anatomi · 9 örnek: Merhaba'dan -O2'ye

Geliştirme ortamını kuralım

Windows · MSYS2

- **UCRT64** terminali:
- `pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain`
- PATH'e: `C:\msys64\ucrt64\bin`

Windows · WSL

- `wsl --install`
- `sudo apt install build-essential gdb`
- ASan ve Valgrind için önerilir

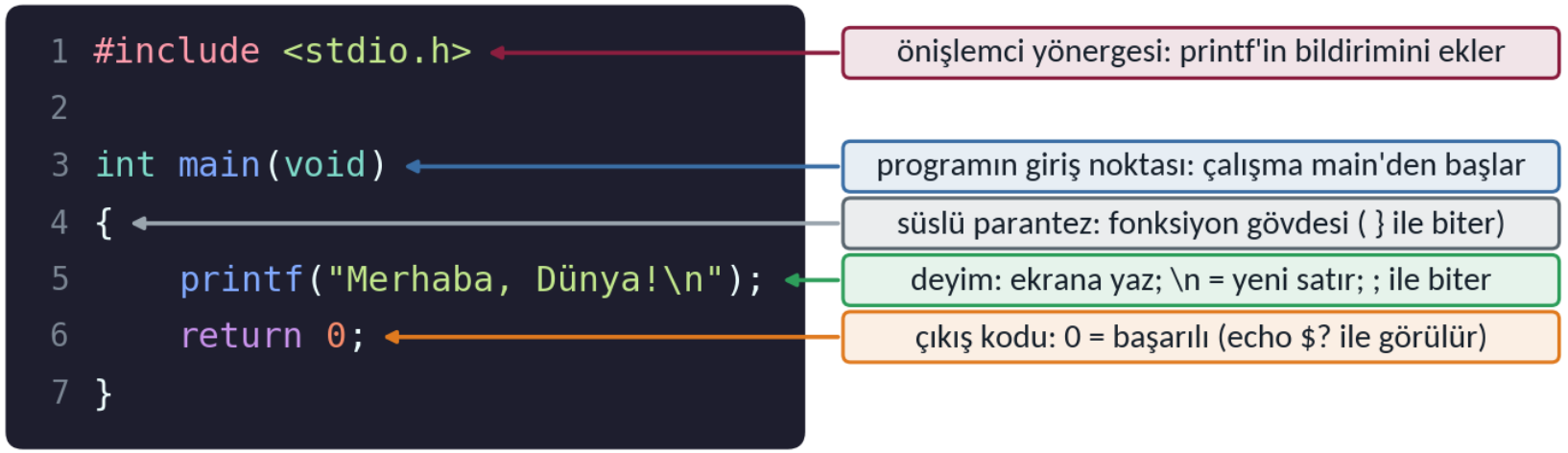
macOS

- `xcode-select --install`
- `gcc` aslında Clang'ı çağırır

Çevrim içi · doğrulama

- godbolt.org · onlinegdb.com
- `gcc --version`
- Türkçe için: `chcp 65001`

İlk C programının anatomisi



- `#include`: önişlemci yönergesi (`;` yok)
- `main`: giriş noktası; `Main` başka bir ad
- `{ }`: gövde; K&R biçimi, 4 boşluk
- Her deyim `;` ile biter
- `return 0;` → çıkış kodu

KOD

Örnek 1 ★ · Merhaba, Dünya!

H01_01_merhaba_dunya.c · satır 4–10

```c
#include <stdio.h>

int main(void)
{
    printf("Merhaba, Dünya!\n");
    return 0;
}
```

ÇIKTI

```
$ gcc -std=c17 -Wall -Wextra -pedantic
    H01_01_merhaba_dunya.c -o merhaba
$ ./merhaba
Merhaba, Dünya!
```

- Derleme **sessizse** hata ve uyarı yok
- `./` = bu klasördeki program
- Windows: `merhaba.exe`
- **Değiştir:** `\n`'yi silin; istem nereye yazılıyor?

KOD

Örnek 2 ★ · Kaçış dizileri

H01_02_kacis_dizileri.c · satır 4–15

```c
#include <stdio.h>

int main(void)
{
    printf("Hafta\tKonu\n");                /* \t : sekme        */
    printf("1\tBilgisayar ve program\n");
    printf("2\tAlgoritmik düşünme\n\n");   /* \n\n : boş satır  */
    printf("Tırnak: \"C dili\"\n");         /* \" : çift tırnak  */
    printf("Yol   : C:\\kodlar\\h01\n");    /* \\ : ters çizgi   */
    printf("Oran  : %%100\n");              /* %% : printf kuralı */
    return 0;
}
```

ÇIKTI

```
Hafta   Konu
1       Bilgisayar ve program
2       Algoritmik düşünme

Tırnak: "C dili"
Yol   : C:\kodlar\h01
Oran  : %100
```

- `\t` → sonraki **sekme durağı**
- `\n\n` → boş satır
- `%%` printf kuralı

Örnek 3 ★ · ASCII sanatıyla isim kartı

H01_03_isim_karti.c · satır 4–17

```c
#include <stdio.h>

int main(void)
{
    printf("+=========================================+\n");
    printf("|   ____                                  |\n");
    printf("|  / ___|   Algoritma ve Programlama - I   |\n");
    printf("| | |       Hafta 1 · İlk C programım      |\n");
    printf("| | |___                                  |\n");
    printf("|  \\____|   Öğrenci : Ada Yılmaz           |\n");
    printf("|           Bölüm   : Bilgisayar Müh.     |\n");
    printf("+=========================================+\n");
    return 0;
}
```

Örnek 3 ★ · Çıktı: hizalamanın sırrı

ÇIKTI (GERÇEK)

```
+=========================================+
|    ____                                 |
|   / ___|   Algoritma ve Programlama - I  |
|  | |       Hafta 1 · İlk C programım     |
|  | |___                                  |
|   \____|   Öğrenci : Ada Yılmaz          |
|             Bölüm   : Bilgisayar Müh.   |
+=========================================+
```

Her satır ekranda **44 sütun**.

Kaynakta `\\____|` → ekranda `\____|`: kaynak satırı **bir karakter uzun**, ekranda eşit.

"Öğrenci" UTF-8'de **9 bayt** (Ö, ğ ikişer), ekranda **7 sütun**: hizalama bayta değil, görünen karaktere göre.

Değiştir: Kartı kendi bilgilerinizle doldurun, bir e-posta satırı ekleyin; sağ kenar hizalı kalsın.

KOD

Örnek 4 ★ · printf ile ilk hesaplama

H01_04_ilk_hesaplama.c · satır 4–13

```c
#include <stdio.h>

int main(void)
{
    printf("3 + 4\n");                  /* metin: olduğu gibi yazılır */
    printf("%d\n", 3 + 4);              /* ifade: önce hesaplanır     */
    printf("%d + %d = %d\n", 12, 30, 12 + 30);
    printf("Bir günde %d saniye var.\n", 24 * 60 * 60);
    return 0;
}
```

- Tırnak içi: **metin**
- Tırnak dışı: önce **hesaplanır**
- %d: tamsayıyı onluk yaz
- Üç %d ↔ üç değer, **sırayla**
- Ayrıntı H3'te

ÇIKTI

```
3 + 4
7
12 + 30 = 42
Bir günde 86400 saniye var.
```

TAHMİN ET

`printf("%d\n", 2 * 60 * 60);` ne yazdırır?

H01_K02_tirnak_ve_hesap.c · satır 11–12

```
11      printf("2 * 60 * 60\n");
12      printf("%d\n", 2 * 60 * 60);
```

A İki kez 2 * 60 * 60

B 2 * 60 * 60 ve 7200

C 7200 ve 7200

D Derleme hatası

Örnek 5 ★ · Noktalı virgülü eksik program

Hatalı (kasıtlı hata)

hatali/H01_05_noktali_virgul.c · satır 1–7

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("Merhaba, Dünya!\n")
6      return 0;
7  }
```

Doğru

H01_01_merhaba_dunya.c · satır 4–10

```
4  #include <stdio.h>
5
6  int main(void)
7  {
8      printf("Merhaba, Dünya!\n");
9      return 0;
10 }
```

Gerçek ileti: `…:5:32: error: expected ';' before 'return'` → kalıp **dosya:satır:sütun: tür: açıklama**

Örnek 6 ★★ · Derleyici mi, bağlayıcı mı?

TERMİNAL (GERÇEK GCC 13 İLETİSİ · PRİNFT)

```
$ gcc -std=c17 -Wall -Wextra -pedantic H01_06a_prinft.c -o program
H01_06a_prinft.c: In function 'main':
H01_06a_prinft.c:5:5: warning: implicit declaration of function 'prinft'; did
    you mean 'printf'? [-Wimplicit-function-declaration]
    5 |     prinft("Merhaba, Dünya!\n");
      |     ^~~~~~
      |     printf
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
H01_06a_prinft.c:(.text+0x18): undefined reference to `prinft'
collect2: error: ld returned 1 exit status
```

DERLEYİCİ · uyarı (warning)
"implicit declaration … did you mean printf?" → devam eder, `.o` üretilir

BAĞLAYICI · hata (error)
`undefined reference to 'prinft'` · `collect2: … ld returned 1` → program **yok**

Örnek 6 ★★ · Main ve nm: adı kim arıyor?

TERMİNAL (GERÇEK ÇIKTI)

```
$ gcc -c ... H01_06a_prinft.c   (uyarı; .o üretildi)
$ nm H01_06a_prinft.o
0000000000000000 T main
                 U prinft
$ gcc H01_06b_main_buyuk_harf.c -o program
/usr/bin/ld: .../Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

Dosya	gcc -c	Bağlama
prinft	uyarı	undefined ref.
Main	sorunsuz	undefined ref. main
#include yok	**2 uyarı**	**başarılı (!)**
doğru program	sorunsuz	başarılı

Algoritma: (1) `gcc -c` ile yalnızca derle → `error` varsa **derleyici**. (2) `.o`'yu bağla → `undefined reference` ise **bağlayıcı**. (3) Çalıştır → yanlışsa **çalışma/mantık**. `nm`: **T** tanımlı, **U** tanımsız ad. MinGW'de `Main` iletisi: “undefined reference to ‘WinMain’”.

KOD

Örnek 7 ★★ · Çıkış kodu

H01_07_cikis_kodu.c · satır 4–10

```c
#include <stdio.h>

int main(void)
{
    printf("Program çalıştı; işletim sistemine 3 döndürüyor.\n");
    return 3;   /* 0: başarılı, 0 dışı: bir sorun oldu */
}
```

ÇIKTI

```
$ ./cikis
Program çalıştı; işletim sistemine 3 döndürüyor.
$ echo $?
3
$ ./cikis > /dev/null && echo "başarılı" || echo "başarısız"
başarısız
```

- `echo $?` → son programın çıkış kodu
- `&&` yalnızca **0**'da, `||` 0 dışında çalışır
- Windows: `echo %errorlevel%`
- Linux'ta 8 bit: `return 256` → **0**

Örnek 8 ★★ · -save-temps: ara dosyalar

TERMİNAL (GERÇEK ÇIKTI · BAYRAKLAR KISALTILDI)

```
$ gcc -save-temps -c H01_08_ara_dosyalar.c
$ gcc H01_08_ara_dosyalar.o -o ara_dosyalar
$ ls -l …   (boyut · ad)
363 H01_08_ara_dosyalar.c
14945 H01_08_ara_dosyalar.i
1520 H01_08_ara_dosyalar.o
691 H01_08_ara_dosyalar.s
15976 ara_dosyalar
$ file H01_08_ara_dosyalar.o ara_dosyalar
….o: ELF 64-bit LSB relocatable, x86-64 …
ara_dosyalar: ELF 64-bit LSB pie executable, x86-64 …
$ ./ara_dosyalar
Merhaba, Dünya!
```

İki adımda derle: önce -c, sonra bağla

.c → .i: 13 satır → 554 satır (stdio.h eklendi, yorumlar silindi)

.o: relocatable (henüz bağlanmadı)

ELF pie executable: çalıştırılabilir

GCC 11+: tek adımda `-save-temps -o ad` ara dosyalara önek ekler.

Örnek 8 ★★ · .s ve .o: derleyicinin ve çeviricinin izi

H01_08_ARA_DOSYALAR.S (KESİT)

```
.string "Merhaba, D\303\274nya!"
main:
  endbr64
  pushq   %rbp
  movq    %rsp, %rbp
  leaq    .LC0(%rip), %rax
  movq    %rax, %rdi
  call    puts@PLT
  movl    $0, %eax
  popq    %rbp
  ret
```

OBJDUMP -D -R H01_08_ARA_DOSYALAR.O (KESİT)

```
8:  48 8d 05 00 00 00 00   lea    0x0(%rip),%rax
b: R_X86_64_PC32  .rodata-0x4
12:  e8 00 00 00 00         call   17 <main+0x17>
13: R_X86_64_PLT32  puts-0x4
```

`printf` → **puts**: biçim belirteci yok ve `\n` ile bitiyor; GCC daha basit fonksiyonu seçti.
`e8 00 00 00 00`: `puts`'un adresi **boş**; `R_X86_64_PLT32` bağlayıcıya "burayı doldur" der.

'ü' = `\303\274` (sekizli) = **C3 BC** · bağlanmış dosyada: `115b: call 1050 <puts@plt>`

Örnek 9 ★★★ · -O0 ile -O2 (Merak)

H01_09_optimizasyon.c · satır 5–14

```c
#include <stdio.h>

int main(void)
{
    int a = 3;
    int b = 4;
    int toplam = a + b;
    printf("Toplam: %d\n", toplam);
    return 0;
}
```

`-O2`'de toplama **yok**: 3 + 4 derlerken hesaplandı → `movl $7, %edx` (sabit katlama).

-O0 · 19 KOMUT

```
endbr64
pushq %rbp
movq %rsp, %rbp
subq $16, %rsp
movl $3, -12(%rbp)
movl $4, -8(%rbp)
movl -12(%rbp), %edx
movl -8(%rbp), %eax
addl %edx, %eax
movl %eax, -4(%rbp)
movl -4(%rbp), %eax
movl %eax, %esi
leaq .LC0(%rip), %rax
movq %rax, %rdi
movl $0, %eax
call printf@PLT
movl $0, %eax
leave
ret
```

-O2 · 10 KOMUT

```
endbr64
subq $8, %rsp
movl $7, %edx
movl $2, %edi
xorl %eax, %eax
leaq .LC0(%rip), %rsi
call __printf_chk@PLT
xorl %eax, %eax
addq $8, %rsp
ret
```

`endbr64`, `__printf_chk`: Ubuntu'nun güvenlik varsayılanları.
Deneyin:
godbolt.org

Hata Avcısı: haftanın hataları

Önce `dosya:satır:sütun`, sonra tür: **error** durdurur, **warning** durdurmaz ama ciddiye alınır. `ld` / `collect2` görüyorsanız sorun **adlarda**.

Hata	İleti (GCC 13)	Aşama	Çözüm
Eksik ; / }	`expected ';' before 'return'`	derleyici	önceki satıra da bak
#include yok	`implicit declaration of 'printf'`	derleyici (uyarı!)	#include <stdio.h>
prinft	`undefined reference to 'prinft'`	bağlayıcı	yazımı düzelt
Main	`undefined reference to 'main'`	bağlayıcı	int main(void)
Bozuk Türkçe	`(ileti yok) D╣nya`	ortam	chcp 65001

Windows konsolunda bozuk Türkçe karakterler

KOD SAYFASI 857 (VARSAYILAN) · TEMSİLÎ

```
C:\ders> merhaba.exe
Merhaba, D╠╝nya!
```

CHCP 65001 SONRASI · TEMSİLÎ

```
C:\ders> chcp 65001
C:\ders> merhaba.exe
Merhaba, Dünya!
```



857 tablosunda **C3** → ╠, **BC** → ╝: iki bayt iki ayrı çizgi karakteri olur. Program doğru; sorun ekranın baytları **yanlış tabloyla** okumasında.

Çözüm: çalıştırmadan önce `chcp 65001` · dosyayı UTF-8 kaydedin (VS Code sağ alt köşe) · Windows Terminal genellikle ek ayar istemez

MÜHENDİS GÖZÜYLE

Mühendis gözüyle: dört alışkanlık

- **Bir altını bilin:** `\n` yoksa tampon boşaltılmaz; çıktı “kaybolur”.
- **Taşınabilirlik:** kaynak taşınır, ikili dosya taşınmaz (ELF ↔ PE, `%rdi` ↔ `%rcx`, x86 ↔ ARM).
- **Uyarılar ücretsiz hata ayıklamadır:** hedef **sıfır uyarı** (`-Wall -Wextra -pedantic`).
- **Tekrarlanabilirlik:** GCC 13 uyarır, GCC 14 durdurur: raporda sürüm, sistem ve bayraklar.
- **Güvenlik:** `endbr64`, `__printf_chk`: derleyicinin sessiz korumaları.

Mini-OBS v0.1: karşılama ekranı ve menü

BEKLENEN EKRAN (REFERANS ÇÖZÜMÜN GERÇEK ÇIKTISI)

```
+================================================+
|   __  __ _       _          ___  ____  ____    |
|  |  \/  (_)_ __ (_)      / _ \| __ )/ ___|    |
|  | |\/| | | '_ \| |_____| | | |  _ \\___ \    |
|  | |  | | | | | | |_____| |_| | |_) |___) |   |
|  |_|  |_|_|_| |_|_|      \___/|____/|____/    |
|                                                |
|     MİNİ-OBS v0.1 · Öğrenci Not Sistemi        |
|     RTEÜ Bilgisayar Mühendisliği · AP-1        |
+================================================+
  ANA MENÜ
  -------------------------------
  [1]  Öğrenci ekle
  [2]  Öğrencileri listele
  [3]  Not gir
  [4]  İstatistikler
  [0]  Çıkış
  -------------------------------
  Seçiminiz: _
  (v0.1: yalnızca görünüm; seçim v0.7'de okunacak.)
```

Bu haftaki adım

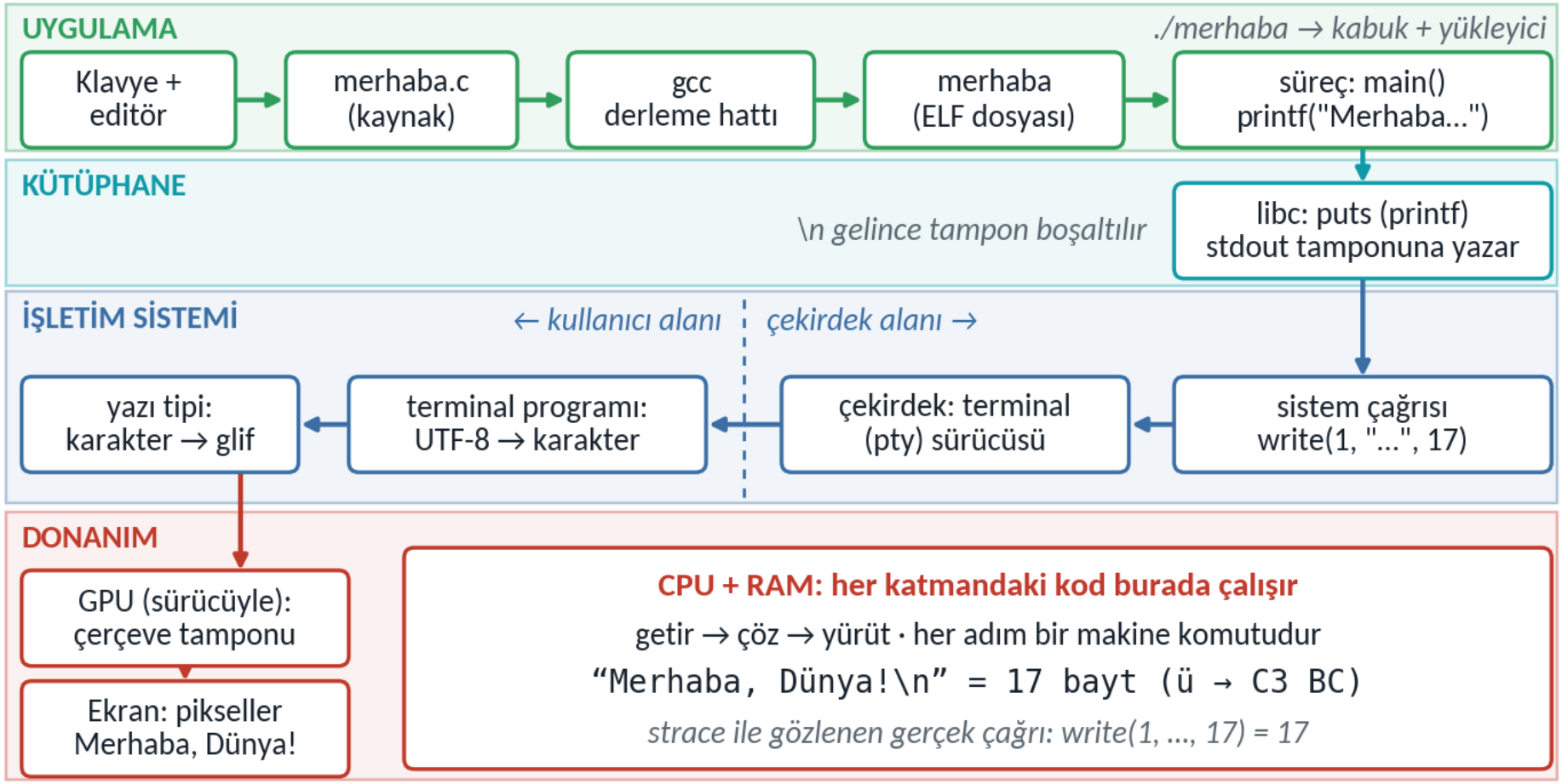
- Kenarları çizili **karşılama kutusu** (ad, sürüm, bölüm)
- **ANA MENÜ**: en az 5 seçenek + “Seçiminiz: _”
- Yalnızca `printf` ve kaçış dizileri
- **Kabul:** sıfır uyarı · hizalı kenar · `echo $?` → 0 · `chcp 65001` ile Türkçe doğru
- Menü = dönem sonu (v1.0) **vizyonu**: seçim ilk kez v0.7'de (H7) okunur (1 Not girişi · 2 Sınıf istatistikleri · 3 Çıkış), sürümlerle büyür

AP-2 KÖPRÜSÜ

AP-2 köprüsü: aynı iş, farklı hız

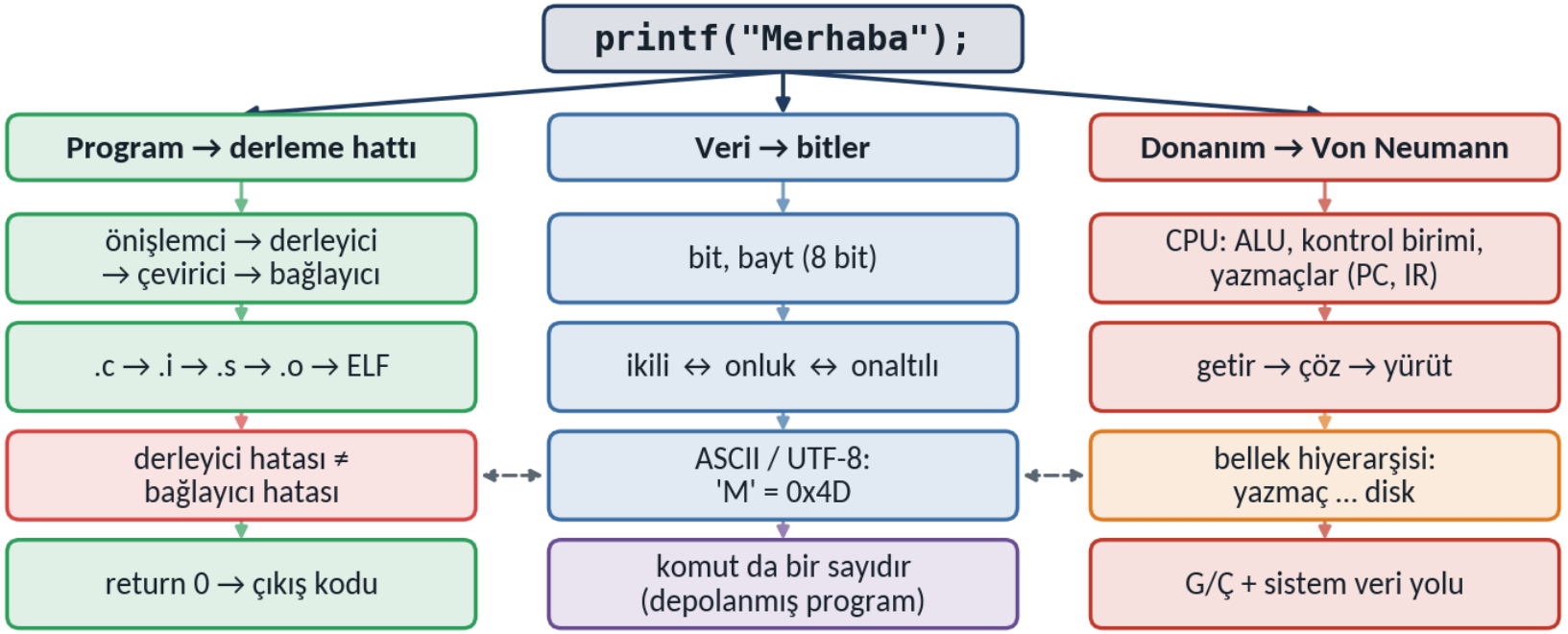
- AP-2'de algoritmaları önce **adım sayısıyla** karşılaştıracaksınız.
- Ama aynı adım sayısı ≠ aynı süre: veri **önbellekteyse** (4 sn) ile **RAM'deyse** (5 dk) arasındaki fark.
- **Önbellek dostu** erişim, algoritmaların pratik performansının temelidir.
- Tohum soru: **Aynı işi yapan iki program neden farklı hızda çalışır?** İlk yanıt: Örnek 9 (eniyileme).
- Dönem boyunca: işlem sayma (H2, H6, H7), `clock()` (H7), önbellek deneyi (H11).

Büyük Soru'nun yanıtı: Merhaba'nın yolculuğu



Editör → gcc hattı → yükleyici → CPU (getir–çöz–yürüt) → libc (puts) → write(1, …, 17) → çekirdek → terminal → yazı tipi → GPU → pikseller

Özet: haftanın kavram haritası



- **Program:** .c → .i → .s → .o → çalıştırılabilir
- **Veri:** her şey bit; ikili ↔ onaltılı; ASCII/UTF-8
- **Donanım:** CPU, bellek, G/Ç; getir–çöz–yürüt
- **Hata:** derleyici ≠ bağlayıcı ≠ ortam
- Ön bilgi yoklamasına dönün!

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. `0x1F` onluk ve ikili tabanda kaçtır?

2. `undefined reference to 'main'` iletisi hangi aşamadan gelir ve olası neden nedir?

3. `printf("A\tB\\n");` ekranda tam olarak ne yazdırır?

Haftaya hazırlık

Okuma

- Bryant ve O'Hallaron (2016), **Bölüm 1**
- Deitel ve Deitel (2022), giriş bölümleri
- Ders notu: Bölüm 3–5

Atölye ve ödev

- Ortamı kur, `gcc --version`
- İsim kartı (Alıştırma 11)
- `-save-temps` raporu (Alıştırma 13)
- Mini-OBS v0.1

Web sayfası

- Oyuncak CPU
- Derleme hattı
- Sayı dönüştürücü
- Quiz: 10 soru

H2'ye köprü

- **Algoritmik düşünme**
- 1 + 2 + … + 100 oyuncak CPU'da kaç komut?
- Gauss bunu nasıl tek işlemle yaptı?